

Söka text i strängar

En vanlig operation som man kan tänkas utföra på en sträng är att söka om en viss substräng finns i en annan sträng, och i så fall på vilket index den börjar. Det finns ett stort antal olika metoder för sökning av substrängar i strängar och följande avsnitt går igenom dessa.

Sökning från början av strängen

Om man behöver hitta en viss substräng i en sträng kan man använda metoden `find()`. Denna returnerar ett giltigt index (av typen `string::size_type`) där strängen börjar om substrängen kunde hittas eller konstanten `string::npos` om substrängen inte kunde hittas. Detta index kan användas som ett normalt heltal.

Man kan som tidigare söka efter `string`, C-strängar och tecken. Ett index kan ges med som en andra parameter. Detta index berättar på vilket index i strängen sökningen skall startas. Några exempel på sökningar:

```
string Text = "The Lord of the Rings";
string S = "Rings";

string::size_type I1 = Text.find ( "Lord" ); // I1 == 4
string::size_type I2 = Text.find ( "lord" ); // I2 == string::npos
string::size_type I3 = Text.find ( ' ' ); // I3 == 3
string::size_type I4 = Text.find ( S, 10 ); // I4 == 16
```

Alla varianter av `find()` stannar vid den första substräng räknat från början som passar. Senare substrängar som även passar ignoreras.

Sökning från slutet av strängen

Sökning från slutet av en sträng fungerar på samma sätt som sökning av en substräng från början av en sträng (se [avsnittet Sökning från början av strängen](#)), med den skillnaden att nu startar sökningen från slutet av strängen. Metoden heter `rfind()` och tar samma argument som vanliga `find()`. Sökningen stannar vid den första hittade substrängen.

Sökning av första och sista förekomst av tecken

Förutom substrängar kan man även söka efter vissa tecken. Detta kan vara praktiskt om man t.ex. behöver dela upp en sträng i delar enligt något tecken. Man kan antingen söka på bas av ett enda tecken eller något tecken i en given sträng. T.ex.:

```
string Text = "The Lord of the Rings";
string S = "ABCL";

string::size_type I1 = Text.find_first_of ( 'e' ); // I1 == 2
string::size_type I2 = Text.find_first_of ( " or" ); // I2 == 3
string::size_type I3 = Text.find_first_of ( S ); // I3 == 4
```

De givna strängarna söks alltså inte i sin helhet, utan de första förekomsten av något av teckne räknas. T.ex. får `I2` värdet 3 trots att texten inte i sin helhet finns i de ursprungliga strängen. Dock finns ' ' (mellanslag) på position 3, så detta index returneras. Den första giltiga positionen returneras.

Man kan även söka efter det *sista* tecknet som finns i strängen som matchar något av de sökta tecknen. Man kan se på detta som att söka från slutet av strängen och returnera den första positionen som passar. Metoden för att göra detta heter `find_last_of()`, och tar samma argument som `find_first_of()`.

Sökning av första och sista förekomst av tecken inte i söksträngen

Det finns två metoder som kan användas för att söka efter tecken i en sträng som *inte* finns i söksträngen. Dessa heter `find_first_not_of()` och `find_last_not_of()`. Dessa söker den första respektive sista positionen i strängen där ett tecken finns som inte finns i söksträngen. Exempel:

```
string Text = "The Lord of the Rings";

string::size_type I1 = Text.find_first_not_of ( "The" ); // I1 == 3
string::size_type I2 = Text.find_first_not_of ( "the" ); // I2 == 0
string::size_type I3 = Text.find_last_not_of ( "Rings" ); // I3 == 15
string::size_type I4 = Text.find_last_not_of ( "abc" );   // I4 == 20
```

[Förutgående](#)

Insättning av text

[Hem](#)[Upp](#)[Nästa](#)

Ersätta text